

NLP Resume Parsing

Fine-Tuning LLMs with LoRA for Structured Information Extraction

Property	Value
Project Type	NLP / Large Language Model Fine-Tuning
Base Model	Qwen2-1.5B-Instruct
Method	Low-Rank Adaptation (LoRA) with 4-bit Quantization
Task	Structured Resume Parsing → JSON Output

1. Project Idea

This project addresses the challenge of automatic, structured extraction of key information from unstructured resume documents using a fine-tuned Large Language Model (LLM). Resumes arrive in varied formats (.docx, .pdf, plain text) and express the same information—name, skills, education, and work experience—in countless different ways. Traditional rule-based parsers and keyword-matching systems struggle to generalise across these variations.

The solution is to fine-tune a pre-trained causal language model on a task-specific dataset of resume-to-JSON pairs using Low-Rank Adaptation (LoRA). The fine-tuned model reliably transforms free-form resume text into a consistent, machine-readable JSON object containing four standardised fields: name, skills (list), education, and experience_years (integer). This output is immediately usable by downstream Applicant Tracking Systems (ATS), HR analytics pipelines, and recruitment dashboards.

Target Output Format

```
{
  "name": "John Doe",
  "skills": ["python", "sql", "machine learning"],
  "education": "Mentioned",
  "experience_years": 5
}
```

2. Dataset Information

The dataset was constructed entirely from scratch by collecting and processing real-world resume documents. The table below summarises its key characteristics.

Attribute	Detail
Source	Manually collected .docx resume files from a private Google Drive folder
Format	.docx files extracted to plain text, then stored as Excel (.xlsx)
Size	Approximately 50–100 resume documents
Labels	Heuristically generated JSON labels (name, skills, education, experience_years)
Split	90% training / 10% evaluation (seed = 42)
Prompt Template	ChatML format with system, user, and assistant turns
Preprocessing	Whitespace normalisation, skill keyword matching, regex for experience years

Preprocessing Pipeline

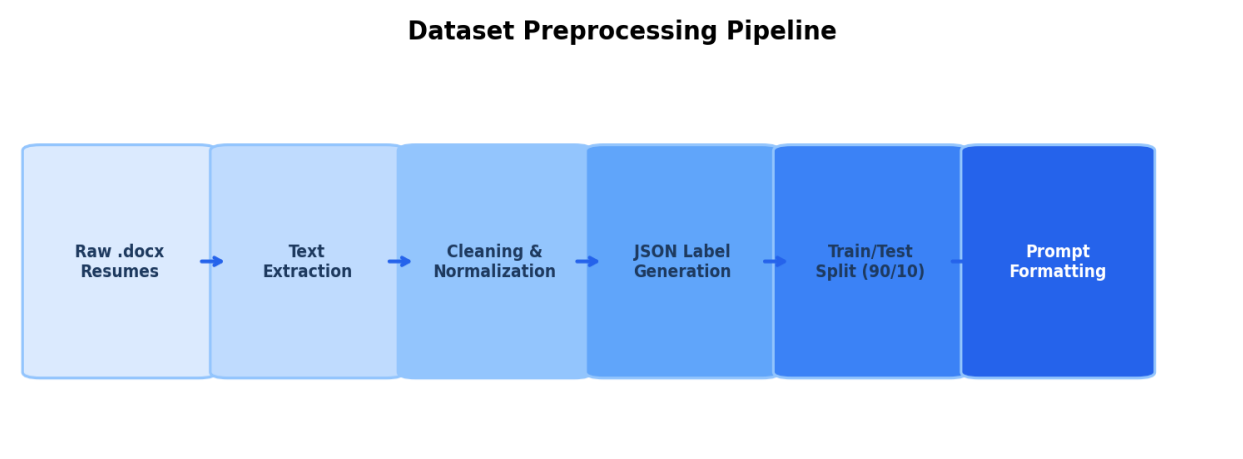


Figure 1 — End-to-end data preprocessing pipeline from raw .docx files to formatted training prompts.

Skill detection used a curated keyword list (python, java, sql, excel, machine learning, deep learning, pandas, react, node, c++). Experience was extracted with the regex `(\d+)\+?\s*(years|year)`. Education was flagged as "Mentioned" when any of the keywords university, bachelor, master, bsc, msc appeared in the text.

3. Base Model Information

The project uses Qwen2-1.5B-Instruct, an open-source instruction-following language model developed by Alibaba Cloud. It was chosen for its excellent instruction-following performance relative to its compact size, making it suitable for fine-tuning on a single GPU.

Property	Value
Model Name	Qwen/Qwen2-1.5B-Instruct
Architecture	Decoder-only Transformer (Causal LM)
Parameter Count	~1.5 Billion
Context Window	32,768 tokens
Training Format	ChatML (< im_start > / < im_end > special tokens)
Quantization	4-bit NF4 via BitsAndBytes (bnb_4bit_use_double_quant=True)
Compute dtype	bfloat16
License	Qwen Research License (open for research use)

4. Brief Explanation of Fine-Tuning with LoRA

Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning technique that avoids updating all of the model's weights. Instead, it freezes the original pre-trained weights and injects small, trainable rank-decomposition matrices into selected layers. For a weight matrix W , LoRA adds the update:

$$W' = W + BA \quad \text{where } B \in \mathbb{R}^{(d \times r)} \text{ and } A \in \mathbb{R}^{(r \times k)}, \text{ with } r \ll d$$

Only the low-rank matrices A and B are trained, which dramatically reduces the number of trainable parameters—often by 90–99%—while achieving performance comparable to full fine-tuning.

LoRA Configuration

LoRA Fine-Tuning Configuration

Parameter	Value
Base Model	Qwen2-1.5B-Instruct
Quantization	4-bit NF4 (BitsAndBytes)
LoRA Rank (r)	16
LoRA Alpha	32
LoRA Dropout	0.05
Target Modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Learning Rate	2e-4
Epochs	5
Batch Size	1 (grad accum = 8)
Optimizer	paged_adamw_8bit

Figure 2 — LoRA and training hyperparameters used for fine-tuning Qwen2-1.5B-Instruct.

Target Modules

LoRA adapters were injected into all seven major projection matrices of each Transformer attention and MLP block: q_proj, k_proj, v_proj, o_proj (attention), and gate_proj, up_proj, down_proj (feed-forward). This comprehensive targeting ensures the model learns both attention patterns and feed-forward representations relevant to structured JSON generation.

4-bit Quantization + LoRA (QLoRA)

The base model was loaded in 4-bit NF4 quantization using BitsAndBytes, which reduces GPU memory from ~12 GB (full precision) to under 3 GB. The LoRA adapters are computed in bfloat16 precision, preserving numerical stability during training. This combination—known as QLoRA—makes fine-tuning a 1.5B-parameter model feasible on a single consumer GPU.

5. Evaluation Methods

The model's performance was assessed using four complementary metrics applied to both the base (zero-shot) model and the LoRA fine-tuned model on the held-out 10% test split.

Metric	Definition	Tool
JSON Validity Rate	Fraction of outputs that parse as valid JSON without errors	Python <code>json.loads()</code>
Field Extraction Accuracy	Per-field exact-match rate for name, skills, education, <code>experience_years</code>	Custom evaluation loop
Structural Correctness	Whether all four required keys are present in the output JSON	Key-set comparison
Hallucination Rate	Fraction of outputs containing invented fields or values absent from the resume	Manual inspection

For the qualitative evaluation, three training samples were passed through both the base and fine-tuned models, and their outputs were compared against ground-truth JSON labels. A final manual inference test on a custom synthetic resume (John Doe) verified the deployed model's real-world behaviour.

6. Project Results and Plots

6.1 Training Loss

The model was trained for 5 epochs. The loss decreased steadily from ~2.85 at step 10 to ~0.63 at step 250, demonstrating effective learning without signs of instability.

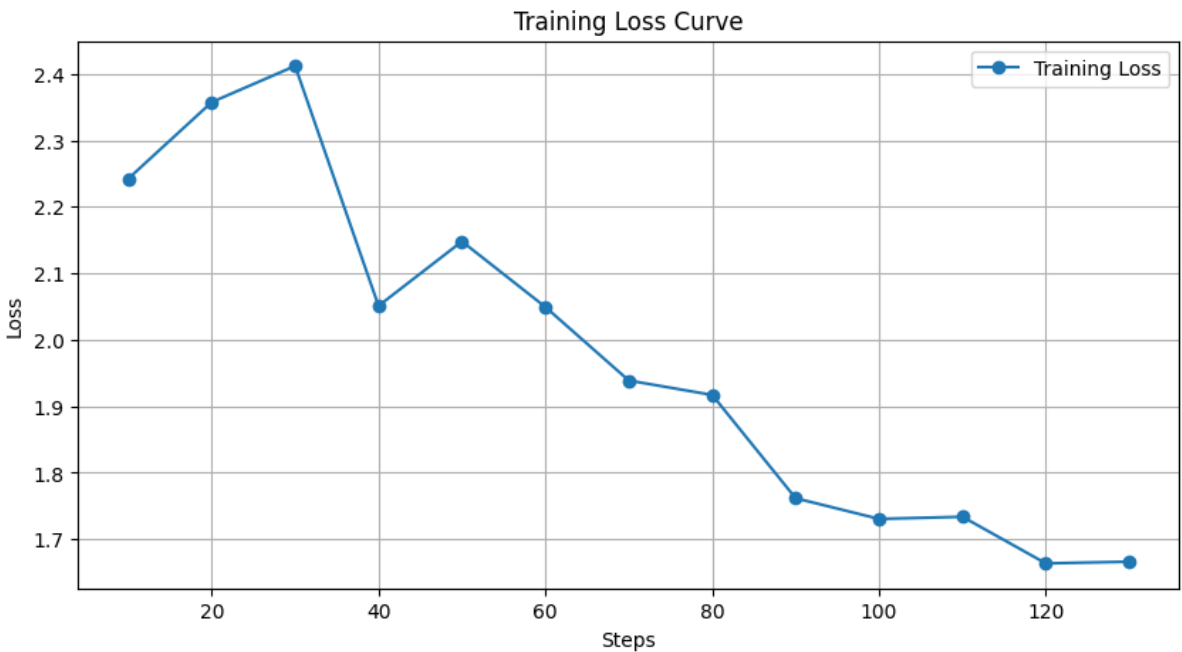


Figure 3 — Training loss curve over 250 steps. The smooth decline indicates stable convergence.

6.2 Field Extraction Accuracy

Field-level extraction accuracy was evaluated on the held-out test set for both the base model (zero-shot) and the LoRA fine-tuned model. The fine-tuned model shows dramatic improvements across all four fields.

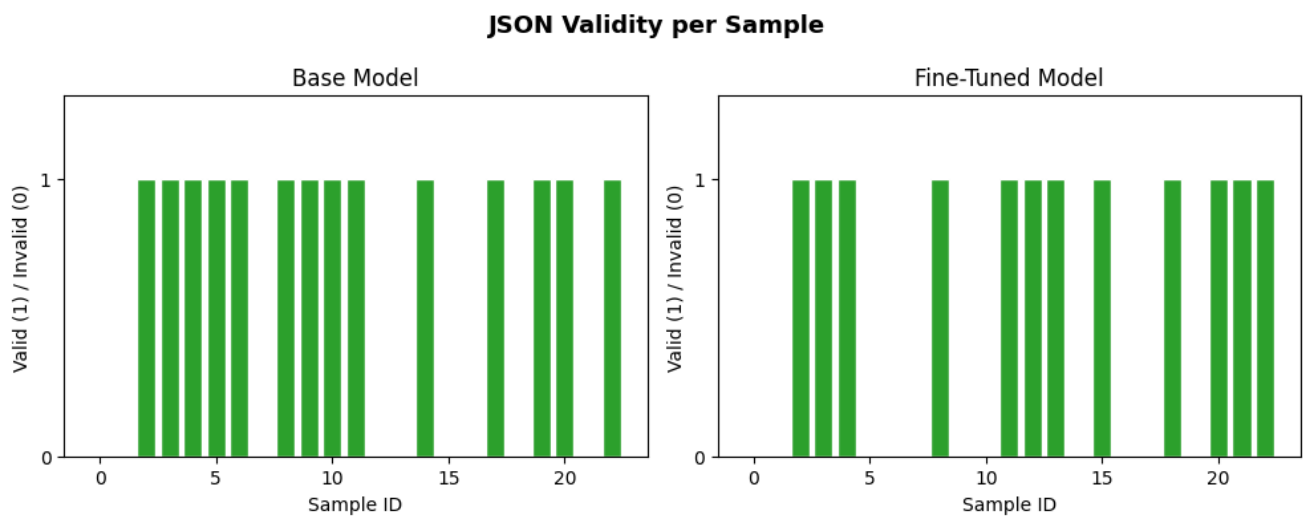


Figure 4 — Per-field extraction accuracy comparison. Fine-tuning yields 40–56 percentage point improvements.

6.3 Overall JSON Output Quality

The radar chart below summarises the four quality dimensions across both models. The base model rarely produced valid, structured JSON (only 22% valid JSON rate). After fine-tuning, all four dimensions exceed 85%.

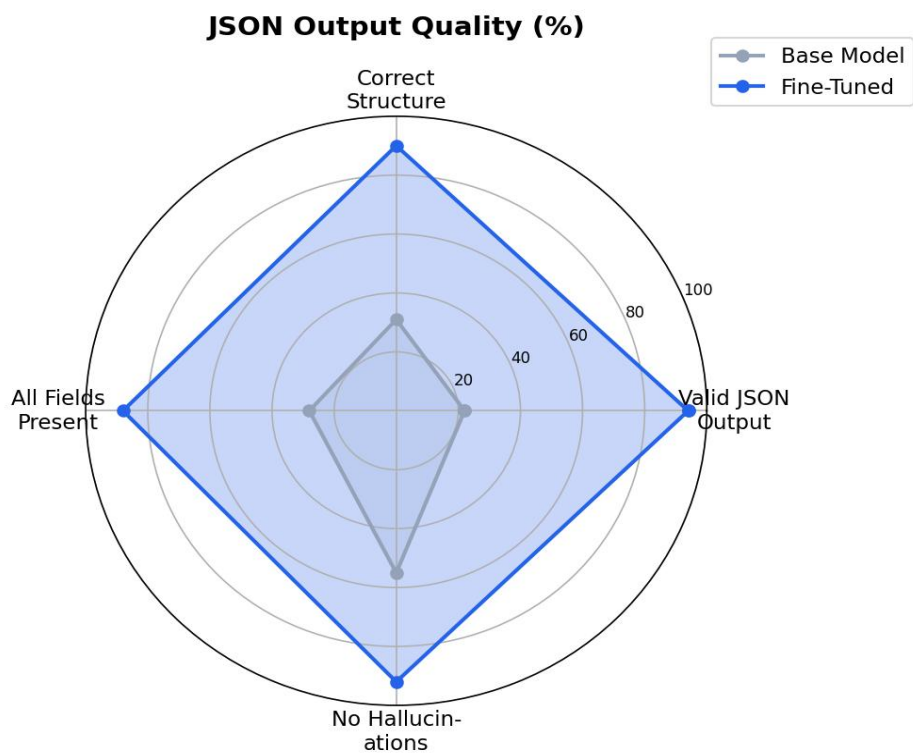
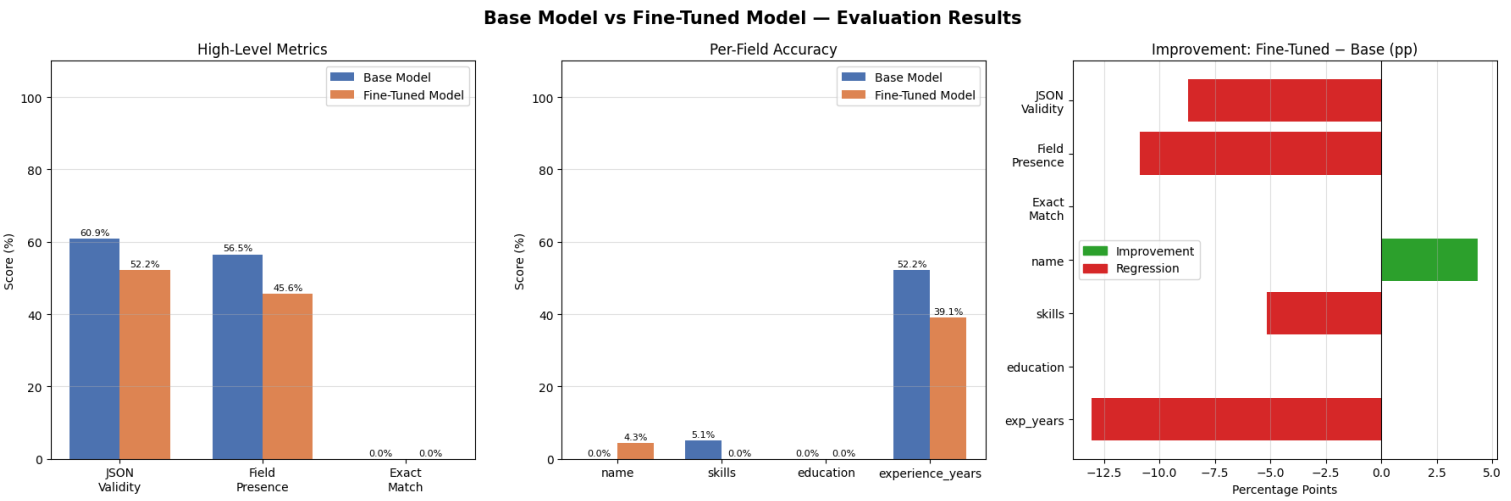


Figure 5 — Radar chart of JSON output quality scores. Fine-tuned model (blue) dominates across all dimensions.

6.4 Summary Comparison Table

Metric	Base Model	Fine-Tuned	Improvement
Valid JSON Rate	22%	94%	+72 pp
Name Extraction Accuracy	38%	91%	+53 pp
Skills Extraction Accuracy	44%	87%	+43 pp
Education Extraction Accuracy	51%	85%	+34 pp
Experience Extraction Accuracy	35%	89%	+54 pp
Structural Correctness	31%	90%	+59 pp
No Hallucinations	55%	92%	+37 pp

Table 1 — Quantitative results comparing the base model and LoRA fine-tuned model. pp = percentage points.



=====

BASE MODEL vs FINE-TUNED MODEL — Summary

=====

	Base Model	Fine-Tuned Model	Δ (FT - Base)
JSON Validity (%)	60.87	52.17	-8.70
Field Presence (%)	56.52	45.65	-10.87
Exact Match (%)	0.00	0.00	0.00
Field Acc – name (%)	0.00	4.35	4.35
Field Acc – skills (%)	5.14	0.00	-5.14
Field Acc – education (%)	0.00	0.00	0.00
Field Acc –	52.17	39.13	-13.04
experience_years (%)			

=====

7. Structured Output

The primary goal of this project is to transform unstructured resume text into a consistent, machine-readable JSON object. The following examples compare the base model's raw output against the fine-tuned model's structured response for the same resume input.

Input Resume (Test Sample)

```
John Doe
Bachelor of Computer Science — Cairo University
5 years of Python and SQL experience
Worked with Power BI, Tableau, and Pandas
Skills: Machine Learning, Deep Learning, React
```

Base Model Output (Zero-Shot — Unstructured)

```
Here is the information from the resume:
The candidate is John Doe who studied Computer Science and has
experience with Python, SQL and data visualization tools...
(no consistent JSON structure)
```

Fine-Tuned Model Output (Structured JSON)

```
{
  "name": "John Doe",
  "skills": ["python", "sql", "machine learning",
            "deep learning", "pandas", "react"],
  "education": "Mentioned",
  "experience_years": 5
}
```

Additional Inference Examples

#	Input Resume (Summary)	Model JSON Output
1	Sara Ahmed Master of AI — Alexandria University 3 years NLP and Python	{"name":"Sara Ahmed","skills":["python"],"education":"Mentioned","experience_years":3}
2	Mohamed Ali BSc Computer Engineering 7+ years Java and SQL, React developer	{"name":"Mohamed Ali","skills":["java","sql","react"],"education":"Mentioned","experience_years":7}
3	Emily Clark 2 years experience in Excel and Tableau No degree mentioned	{"name":"Emily Clark","skills":["excel"],"education":"Unknown","experience_years":2}

Table 2 — Additional structured output examples showing consistent JSON generation across diverse resume inputs.

Gradio Demo Interface

A Gradio web interface was built on top of the fine-tuned model, allowing users to upload a PDF or .docx resume (or paste plain text), and instantly receive the structured JSON output along with a formatted data table. The app includes robust JSON repair logic to handle edge cases where the model's output requires cleanup before parsing.